

Fundamentos de Teoría de la Computación

Parciales viejos - 1er parcial - Computabilidad y Complejidad

2024	2
Ejercicio 1	2
Ejercicio 2	2
Ejercicio 3	2
Ejercicio 4	3
Ejercicio 5	3
Ejercicio 6	3
Ejercicio 7	3
Ejercicio 8	4
Ejercicio 9	4
Ejercicio 10	4
2023	6
Ejercicio 1	6

2024

Ejercicio 1

a) *Definir cuándo un lenguaje es recursivo*

Un lenguaje L es recursivo si existe una máquina de Turing que lo acepta y para siempre

b) *Dar un ejemplo de un lenguaje recursivo y un ejemplo de un lenguaje no recursivo*

Un lenguaje recursivo podría ser $SAT = \{\phi \mid \phi \text{ es una fórmula booleana sin cuantificadores satisfactible}\}$

Un lenguaje no recursivo podría ser $HP = \{\langle M \rangle, w \mid \text{la MT } M \text{ se detiene a partir de } w\}$

c) *¿Si un lenguaje es recursivo, también lo es su complemento? Justificar*

Sí, esto se debe a que si L es decidido por una MT M , se puede construir una MT M' en base a M , que cuando M hace q_A , M' haga q_R y cuando M hace q_R , M' haga q_A . De esta manera M' acepta L^c y como M siempre para, M' también lo hará

Ejercicio 2

Dadas dos MT M_1 y M_2 que reconocen lenguajes L_1 y L_2 de la clase RE, respectivamente. Explicar cómo se puede construir una MT M que reconozca la intersección entre M_1 y M_2 . Comentario: sólo dar la idea general

Para realizar esto, habría que ejecutar primero M_1 , si rechaza o loopea, M rechaza o loopea, ya que la cadena de entrada, al no ser aceptada por M_1 no está en la intersección entre L_1 y L_2 . Si M_1 acepta, M ejecuta M_2 y hace lo que M_2 haga; si acepta, entonces la cadena de entrada está en la intersección entre ambos lenguajes, por lo que M acepta, y si M_2 rechaza o loopea, M también lo hará ya que la cadena no está en la intersección

Ejercicio 3

Dado el lenguaje $D = \{w_i \mid M_i \text{ acepta } w_i\}$, considerando el orden canónico de las cadenas, explicar cómo se puede construir una MT M que lo reconozca. Comentario: sólo dar la idea general

Para que M reconozca a ese lenguaje primero debería generar todas las cadenas en orden canónico, manteniendo un contador, hasta encontrar la cadena de entrada w_i , al hacer eso, M sabría cuál es el valor de i , por lo que debería generar todas las cadenas en orden canónico que correspondan a MTs válidas hasta llegar a la MT M_i . Y luego ejecuta sobre M_i w_i , y retorna lo que M_i retorne (si acepta M acepta, si rechaza o loopea, M también lo hace)

Ejercicio 4

Se cumple que todos los lenguajes de RE se reducen al lenguaje HP. Explicar por qué, si HP fuera recursivo, se cumpliría $RE = R$

Esto es así porque, en una reducción $L_1 \leq L_2$, por las propiedades de la reducción, si $L_2 \in R$, entonces L_1 pertenece a R , y dado que todos los lenguajes de RE se reducen a HP, si el mismo perteneciera a R , entonces todos los lenguajes de RE lo harían, haciendo que se cumpla $RE = R$

Ejercicio 5

Explicar por qué la función $f: \Sigma^ \rightarrow \Sigma^*$ no es una reducción del lenguaje L_U al lenguaje L_U^c : a todo par $(\langle M \rangle, w)$ le asigna el par $(\langle M^c \rangle, w)$, tal que $\langle M^c \rangle$ es como $\langle M \rangle$, salvo que sus estados finales están invertidos, es decir, donde aparece q_A en $\langle M \rangle$ aparece q_R en $\langle M^c \rangle$ y viceversa. Comentario, para simplificar asumir que los pares $(\langle M \rangle, w)$ de entrada siempre son sintácticamente correctos*

La función f no es una reducción de L_U a L_U^c , porque para que se cumpla eso, debería ser total computable (cosa que sucede) y para todo elemento w , si $w \in L_U$, $f(w) \in L_U^c$, y si $w \notin L_U$, $f(w) \notin L_U^c$. El problema es que en los casos donde $w \notin L_U$, como $L_U \in RE$, entonces $\langle M \rangle$ podría hacer q_R o looppear, en el caso de que $\langle M \rangle$ loopee, $\langle M^c \rangle$ también lo hará, por lo que dicha cadena de entrada w pertenece a L_U^c , ya que $\langle M^c \rangle$ no la acepta, por lo que la función f no es una reducción de L_U a L_U^c

Ejercicio 6

a) *Defina cuando un lenguaje pertenece a la clase P*

Un lenguaje L pertenece a P si existe una MT M que lo decide en tiempo polinomial con respecto a su entrada

b) *Dar un ejemplo de un lenguaje de P y un ejemplo de un lenguaje que no estaría en P, justificando por qué no estaría*

Un lenguaje de P sería $L_P = \{w \mid w \text{ es un palíndromo con símbolos } a \text{ y } b\}$

Un lenguaje que no estaría en P sería $SAT = \{\phi \mid \phi \text{ es una fórmula booleana sin cuantificadores satisfactible}\}$, ya que para poder decidir si ϕ es una fórmula booleana sin cuantificadores satisfactible, se deberían evaluar todas las posibles asignaciones de valores verdad para ϕ , y esta cantidad es 2^x , donde x es la cantidad de variables que tiene ϕ , y como x depende del tamaño de la entrada, entonces la MT que decida SAT lo hará en tiempo $O(2^n)$, donde $n = |w|$, es decir, $\exp(n)$

Ejercicio 7

a) *Defina cuando un lenguaje pertenece a la clase NP*

Un lenguaje L pertenece a NP si existe una MT M que lo verifica en base a un certificado x , en tiempo $\text{poly}(n)$

- b) *Dar un ejemplo de un lenguaje de NP e indicar cómo serían los certificados de sus cadenas*

Un lenguaje de NP sería $CH = \{G \mid G \text{ es un grafo con un circuito de Hamilton}\}$.
Sus certificados deberían ser un conjunto de vértices

Ejercicio 8

Sea L_1 un lenguaje NP-completo y L_2 un lenguaje de NP. Explicar por qué, si existe una reducción polinomial de L_1 a L_2 , se cumple que L_2 es NP-completo

Esto es así porque sabemos que L_1 es NP-completo, entonces eso significa que está en NP y es NP-difícil, es decir, todos los lenguajes de NP se reducen a él. Y como la reducción polinomial es transitiva, entonces, existirá una reducción polinomial de todos los lenguajes de NP a L_2

Ejercicio 9

Sea TAUT el lenguaje de las tautologías, es decir: $TAUT = \{\varphi \mid \varphi \text{ es una fórmula booleana sin cuantificadores satisfactible con toda asignación de valores de verdad}\}$.

- a) *Explicar por qué TAUT no estaría en la clase P*

TAUT no estaría en P porque poder decidirlo, se deberían evaluar todas las posibles asignaciones de valores verdad para φ , y esta cantidad es 2^x , donde x es la cantidad de variables que tiene φ , y como x depende del tamaño de la entrada, entonces la MT que decida TAUT lo hará en al menos tiempo $O(2^n)$, donde $n = |w|$, es decir, $\exp(n)$, por lo que TAUT no estaría en P

- b) *Explicar por qué TAUT no estaría ni siquiera en la clase NP*

TAUT no estaría en NP ya que su certificado no es sucinto. Esto es así porque el mismo debería tener un tamaño exponencial con respecto a la entrada ya que debería incluir todas las asignaciones de valores de verdad posibles, y como vimos, esto es de orden $\exp(n)$, por lo que la verificación de TAUT en base a un certificado x no sería polinomial ergo, TAUT no estaría en NP

Ejercicio 10

- a) *¿Qué lenguajes se consideran tratables en la jerarquía espacial? Justificar por qué*

Los de la clase LOGSPACE, es decir, que tienen una MT M que al ejecutarse desde toda entrada w , ocupa a lo sumo espacio logarítmico con respecto a su entrada. Estos lenguajes son tratables ya que al tener espacio acotado, la cantidad de configuraciones posibles que podrá tener M será del orden de $c^{\log(n)}$, es decir, $n^{\log(c)}$, es decir, $\text{poly}(n)$, por lo que los lenguajes de LOGSPACE están también en P

- b) *Explicar por qué una MT que ocupa espacio $\text{poly}(n)$ no tiene por qué tardar tiempo $\text{poly}(n)$*

Esto es así porque la cantidad de configuraciones posibles que hay para una MT que ocupa espacio $\text{poly}(n)$ es de $c^{\text{poly}(n)}$, es decir, $\exp(n)$, por lo que la MT podría no tardar tiempo $\text{poly}(n)$

- c) *Dada una MT M_1 que ocupa espacio $S(n)$, explicar cómo se puede construir una MT M_2 que en espacio $S(n)$ imprima una cadena con $S(n)$ marcas X*

Para hacer eso, la MT M_2 debería ejecutar en espacio $S(n)$ a M_1 y para cada una de sus cintas debería imprimir una marca X , imprimiendo al finalizar $S(n)$ marcas

2023

Ejercicio 1

MT $M = (Q, \Gamma, \delta, q_0, q_A, q_R)$

$Q = \{q_0, q_1, q_2, q_3, q_A, q_R\}$

$\Gamma = \{B, 1, 2, 3\}$

δ :

$(q_0, B) = (q_A, B, S)$

$(q_0, 1) = (q_2, 1, R)$

$(q_2, 2) = (q_3, 2, R)$

$(q_3, 3) = (q_1, 2, R)$

$(q_1, 1) = (q_2, 1, R)$

$(q_1, B) = (q_A, B, S)$